



We are on a mission to address the digital skills gap for 10 Million+ young professionals, train and empower them to forge a career path into future tech

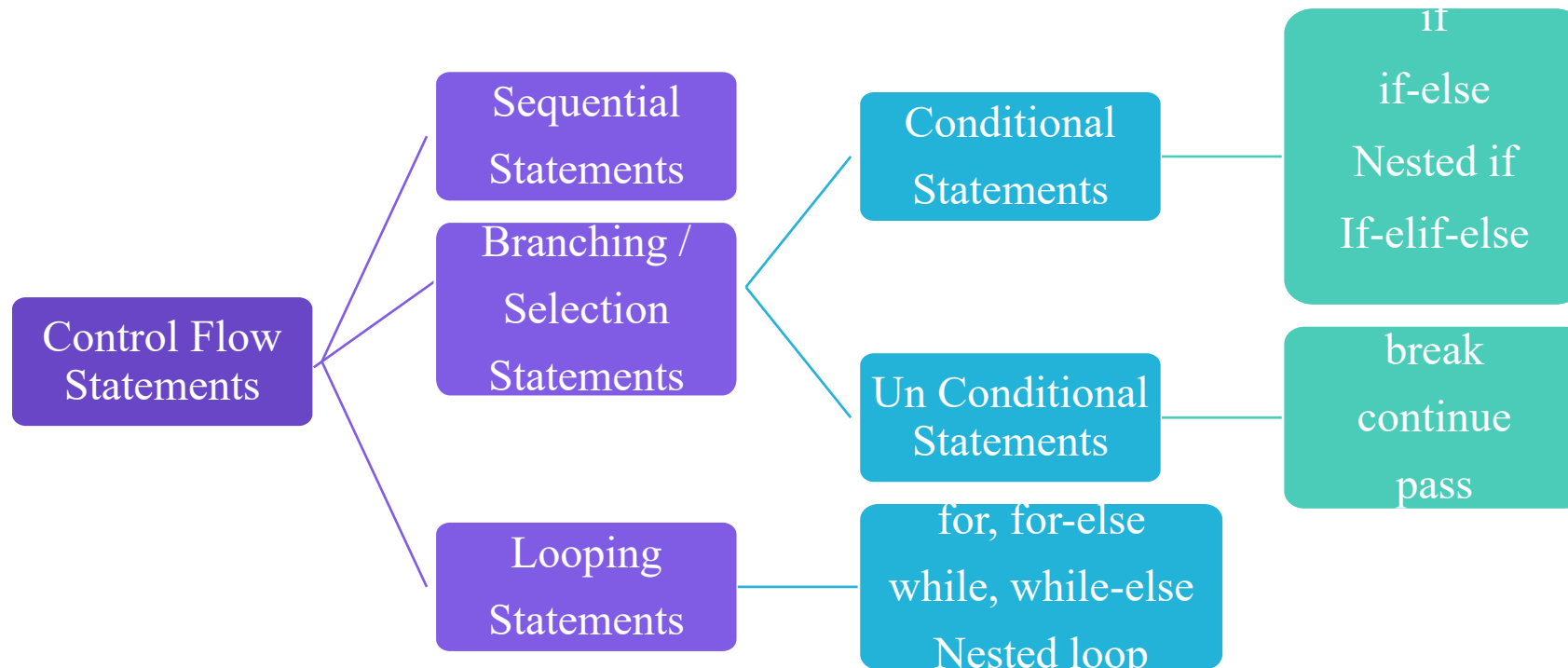
Control Flow Statements

MAY, 2026



Introduction

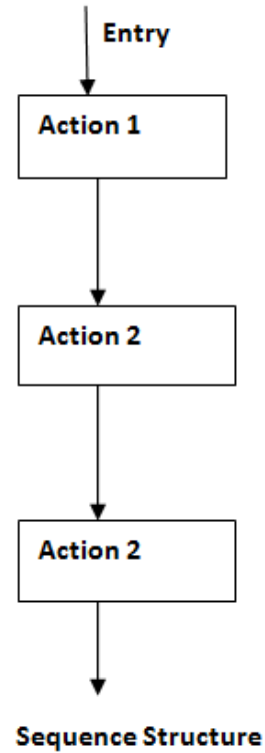
- In a program, the control flow is the **order in which the program's code executes**. Control statements are elements in the source code that **control the flow of program execution**.



Control Flow Statements

Sequencing (Action)

- This describes a sequence of actions that a program carries out **one after another**, unconditionally.
- Execute a list of **statements in order**



Sequencing (Action)

Example 1: To get employee details, wages and number of days worked from user and find total salary.

```
# Calculate total salary from wages and no of days worked
```

```
Name=input("Enter Employee Name:\n")
```

```
Emp_Id=int(input("Enter Employee Id:\n"))
```

```
No_of_days=int(input("Enter No Days Present:\n"))
```

```
Wages=int(input("Enter wages per Day:\n"))
```

```
Total_Salary= No_of_days * Wages
```

```
print("Employee Details:\n")
```

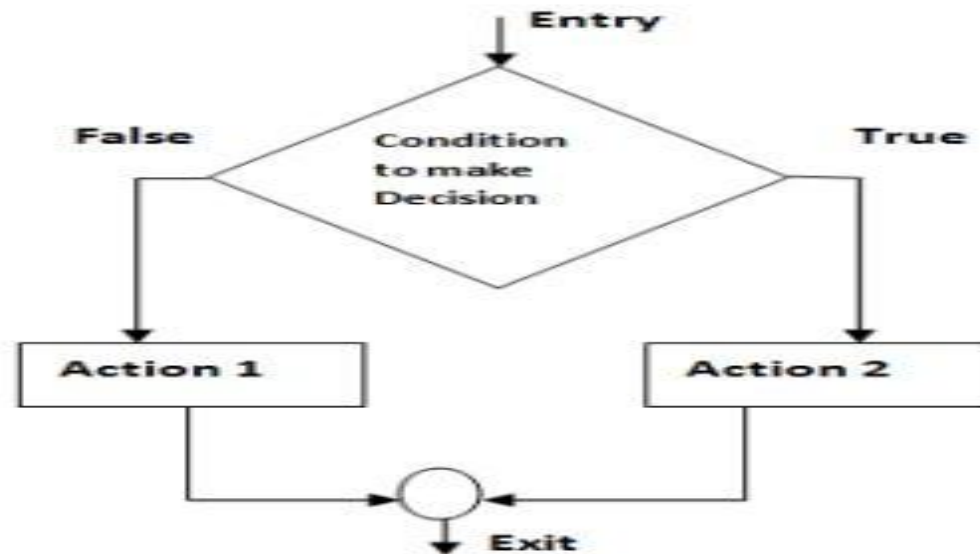
```
print("Employee Name:", Name, end='\n');
```

```
print("Employee ID:", Emp_Id, end='\n');
```

```
print("Total Salary:", Total_Salary, end='\n');
```

Selection (Decision)

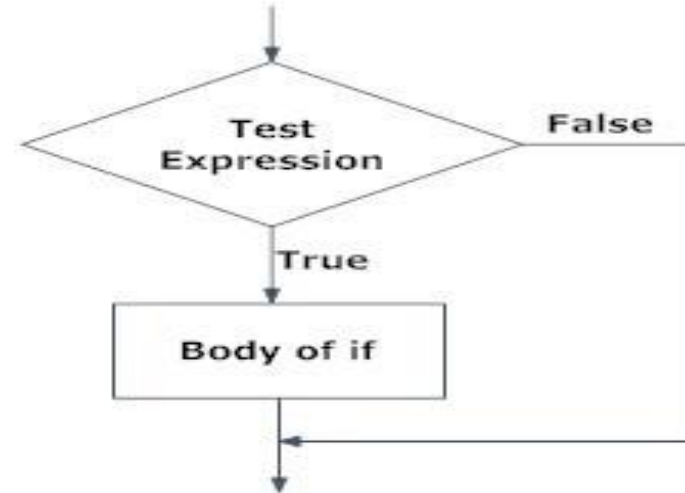
- **Conditional Statements** also known as **selection statements**, are used to make **decisions** based on a **given condition**.
- If the condition evaluates to **True**, a **set of statements** is executed, **otherwise another set** of statements is executed.



Selection: Simple If Statement

- The if statement **evaluates the test expression**, If test expression is **evaluated to true**, statements **inside the body** of if is executed. If test expression is **evaluated to false**, statements **inside the body** of if is skipped.
- **Syntax:**

```
if condition :  
    indentedStatementBlock
```



Selection: Simple If Statement

Example 1:

#Checking the vehicle speed using simple if

```
Moving_limit=60
```

```
currentspeed=int(input("enter the speed"))
```

```
if currentspeed > Moving_limit:
```

```
    print("Over speed ,need to be controlled")
```

```
print("Happy journey")
```

Note:

In Python, the body of the if statement is indicated by the indentation.

Selection: Simple If Statement

Example 2:

To increment a given number if it is positive

```
number=int(input("Enter any number:\n"))
```

```
if number>0:
```

```
    number=number+1
```

```
    print("The incremented number is:",number)
```

Selection: If-else Statement

- **Alternative Statements** are also known as **If-Else Statement**, in which the if statement evaluates the test expression.
- If test expression is **evaluated to true**, statements inside the **body of if** is executed.
- If test expression is **evaluated to false**, statements **inside the body of else** is executed.
- **Syntax:**

Syntax

if condition:

 indentedStatementBlockForTrueCondition

else:

 indentedStatementBlockForFalseCondition

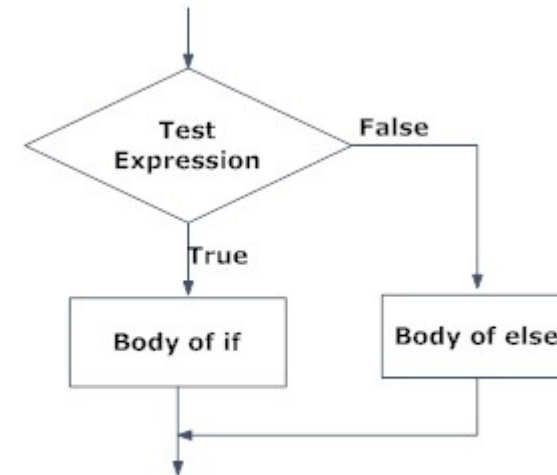


Fig: Operation of if...else statement

Selection: If-else Statement

Example 1:

#Checking the vehicle speed using if-else

Moving_limit=60

currentspeed=int(input("enter the speed"))

if currentspeed > Moving_limit:

 print("Over speed ,need to be controlled")

else:

 print("The speed is up to the limit")

print("Happy journey")

Selection: If-else Statement

Example 2:

#To check whether the given number is odd or even number

```
number = int(input("Enter a number:\n"))
```

```
if (number % 2) == 0:
```

```
    print(number," is Even number")
```

```
else:
```

```
    print(number," is Odd number")
```

Selection: If - elif - else Statement

- **Chained conditional Statements** are also known as **if–elif-else Statement**, in which the “**if Statement**” executes the statements only if the condition is **True**.
- “**elif Statement**” provides an alternative possibility for the “**if Statement**”. The execution of “**Elif Statement**” is same as the “**if Statement**” (i.e) “**elif Statement**” executes the statements only if the condition is **True**.
- If the condition given in the “**elif Statement**” is also **False**, then “**else Statement**” gets executed.

Selection: If - elif - else Statement

- **Syntax:**

```
if condition1 :  
    indentedStatementBlockForTrueCondition1  
elif condition2 :  
    indentedStatementBlockForFirstTrueCondition2  
elif condition3 :  
    indentedStatementBlockForFirstTrueCondition3  
elif condition4 :  
    indentedStatementBlockForFirstTrueCondition4  
else:  
    indentedStatementBlockForEachConditionFalse
```

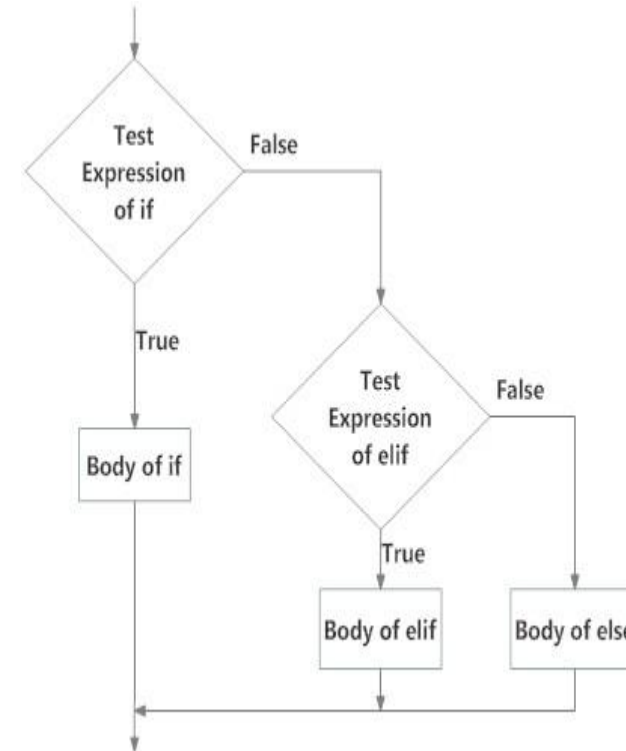


Fig: Operation of if...elif...else statement

Selection: If - elif - else Statement

Example 1:

#To test whether a number entered by the user is negative / positive / zero

```
num=int(input("Enter any number:\n"))
```

```
if num > 0:
```

```
    print(num,"is Positive number")
```

```
elif num == 0:
```

```
    print(num,"is Zero")
```

```
else:
```

```
    print(num,"is Negative number")
```

Selection: If - elif - else Statement

Example 2:

"""Display the grade of a student based on his/her Python subject marks

Marks	Grade	Marks	Grade
>90	O	81 to 90	A
71 to 80	B	61 to 70	C
50 to 60	D	< 50	F"""

```
name=input("Enter your Name\n")
mark=int(input("Enter your Python Mark\n"))
if mark>90:
    print("The equivalent grade of",mark,"is O")
elif mark>80 and mark<=90:
    print("The equivalent grade of",mark,"is A")
elif mark>70 and mark<=80:
    print("The equivalent grade of",mark,"is B")
elif mark>60 and mark<=70:
    print("The equivalent grade of",mark,"is C")
```

Selection: If - elif - else Statement

Example 2:

"""Display the grade of a student based on his/her Python subject marks

Marks	Grade	Marks	Grade
>90	O	81 to 90	A
71 to 80	B	61 to 70	C
50 to 60	D	< 50	F ""

elif mark>=50 and mark<=60:

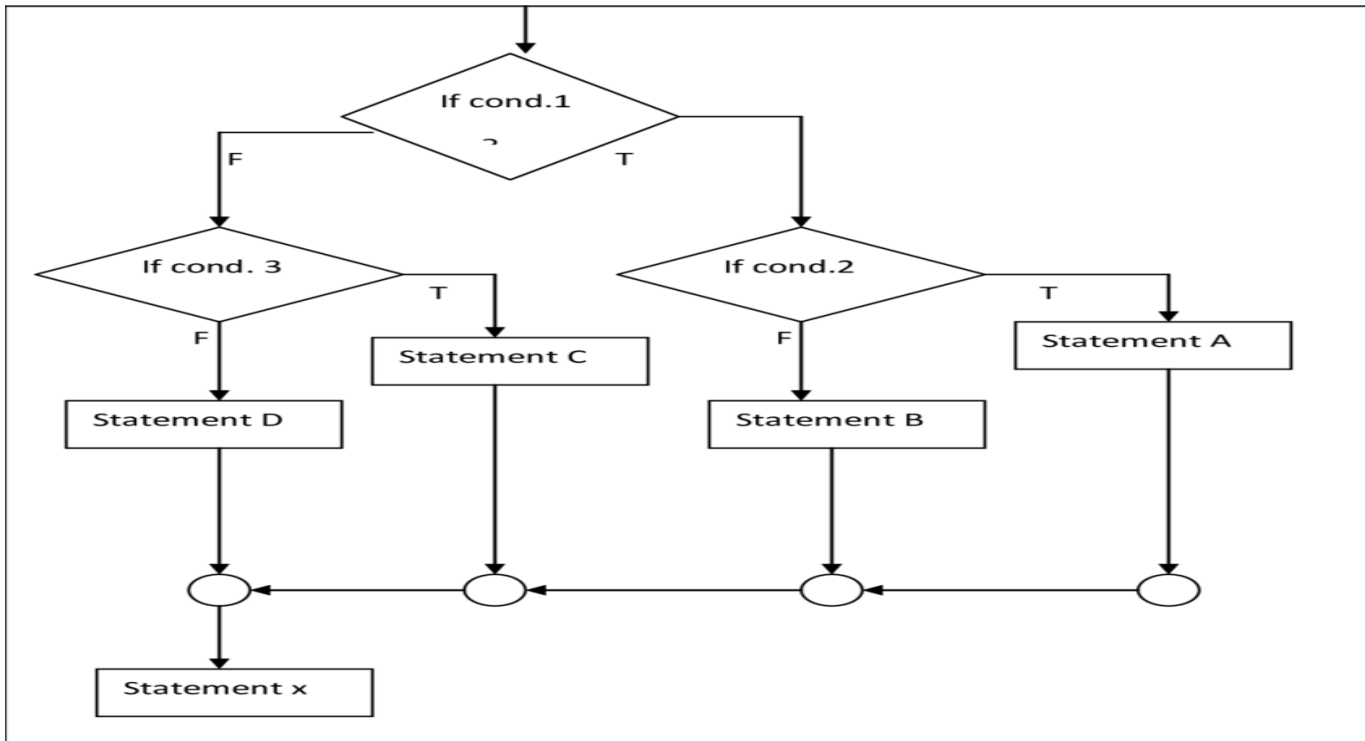
 print("The equivalent grade of",mark,"is D")

else:

 print("The equivalent grade of",mark,"is F")

Selection: Nested If Statement

- Any number of if-else statements can be **nested inside one another** i.e. can use one if or else if statement inside another if or else if statement(s).
- Indentation** is the only way to figure out the **level of nesting**.



Selection: Nested If Statement

Example 1:

#To find the greatest among three numbers

```
a=int(input("Enter the first number: "))
```

```
b=int(input("Enter the second number: "))
```

```
c=int(input("Enter the third number: "))
```

```
if a>b:
```

```
    if a>c:
```

```
        print(a,"is the greatest number than",b,"and",c)
```

```
    else:
```

```
        print(c,"is the greatest number than",a,"and",b)
```

```
elif b>c:
```

```
    print(b,"is the greatest number than",a,"and",c)
```

```
else:
```

```
    print(c,"is the greatest number than",a,"and",b)
```

Selection: Nested If Statement

Example 2:

#To Read a Coordinate Point in a XY Coordinate System and Determine its Quadrant

```
X=int(input('Enter the values for X: '))
```

```
Y=int(input('Enter the values for Y: '))
```

```
if X > 0:
```

```
    if Y > 0:
```

```
        print ('X, Y point lies in the First quadrant ')
```

```
    else:
```

```
        print ('X, Y point lies in the Fourth quadrant ')
```

```
elif Y<0:
```

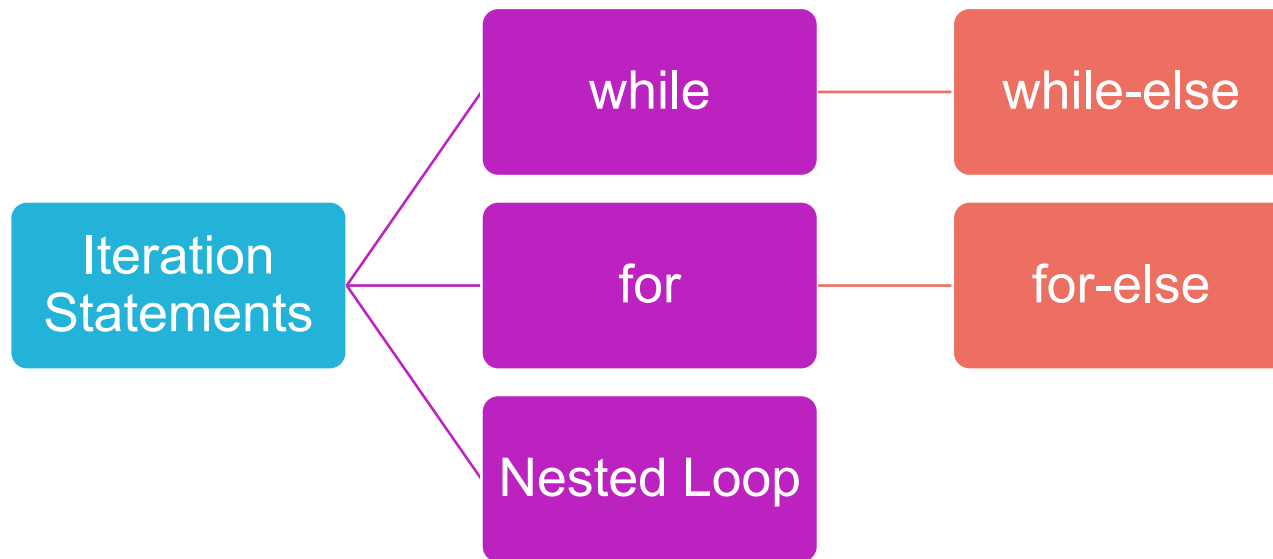
```
    print ('X, Y point lies in the Third quadrant ')
```

```
else:
```

```
    print ('X, Y point lies in the Second quadrant ')
```

Iteration (Looping)

- **Iteration** is a process of executing the set of **statements repeatedly until** the condition is **True**.
- It is also known as **looping statements or repetition**.



Iteration: while Loop

- **While loop statement** is a set of statements that are repeatedly executed only if the condition given in while loop is **True**. When the condition becomes False, then the loop gets terminated.
- The while loop in Python is **used in** situations where **we do not know the exact number of iterations of loop beforehand**. The loop execution is terminated on the basis of the test condition.
- **Syntax:**

```
while condition :  
    indentedBlock
```

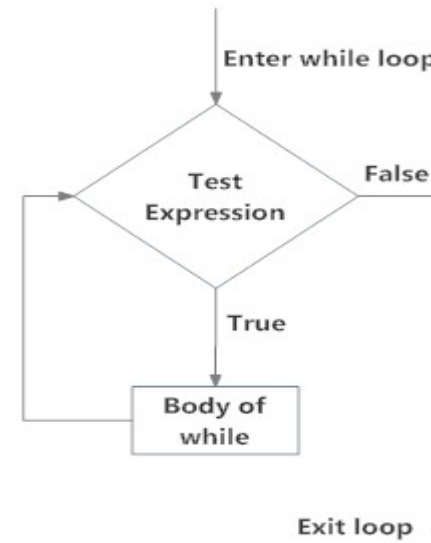


Fig: operation of while loop

Iteration: while Loop

Example 1:

#Print first N natural numbers.

```
N=int(input('Enter the value of N: '))
```

```
counter=1    # initialization
```

```
while counter<=N:  # Condition
```

```
    print(counter)
```

```
    counter=counter+1    # Increment
```

```
print('Exit from loop')
```

Trace the Program

N=3

counter=1

1<=3 (True)

print 1

counter=2

2<=3(True)

print 2

counter=3

3<=3(True)

print 3

counter=4

4<=3 (False)

Print Exit from loop

Iteration: while Loop

Example 1:

```
#Calculate the sum of first N natural numbers
N=int(input('Enter the value of N: '))
#Initialize counter and sum variable
counter=1
sum=0
while counter<=N: # Condition
    sum=sum+counter
    counter=counter+1 # Increment
print('Sum is: ',sum)
```

Trace the Program

```
N=3
counter=1
sum=0
1<=3 (True)
    sum=0+1=1
    counter=2
2<=3(True)
    sum=1+2 =3
    counter=3
3<=3(True)
    sum=3+3=6
    counter=4
4<=3 (False)
Sum is: 6
```

Iteration: for Loop

- **For loop statement** is a set of statements that are **repeatedly or iteratively executed** over a sequence. When the **condition becomes False**, then the loop gets **terminated**.

- **Syntax:**

for variable in sequence:
statement

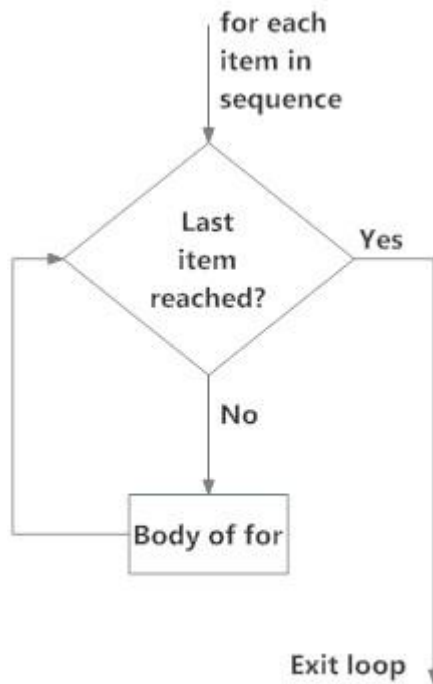


Fig: operation of for loop

Iteration: for Loop

range()

- If you do need to iterate over a **sequence of numbers**, the **built-in function range()** comes in handy.

- **Syntax:**

`range(start, stop, step_size).`

Note: `step_size` defaults to 1 if not provided.

Example 1:

range(10) or **range(0,10)** will generate numbers from 0 to 9 (10 numbers).

range(5, 10) : 5, 6, 7, 8, 9

range(0, 10, 3) : 0, 3, 6, 9

range(-10, -100, -30): -10, -40, -70

Iteration: for Loop

Example 1:

#Calculate the factorial of a given number

Note: In mathematics, the factorial of a positive integer n , denoted by $n!$, is the product of all positive integers less than or equal to n .

Example: $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$.

```
N = int(input("Enter a number: ")) # To take input from the user
factorial = 1
if N < 0: # check if the number is negative, positive or zero
    print("Sorry, factorial does not exist for negative numbers")
elif N == 0:
    print("The factorial of 0 is 1")
else:
    for i in range(1, N + 1):
        factorial = factorial*i
    print("The factorial of",N,"is",factorial)
```

Iteration: for Loop

Example 2:

#Print multiplication table of a given number

```
N=int(input("Display multiplication table of "))
```

```
Limit=int(input('Enter the limit: '))
```

```
for i in range(1,Limit+1):
```

```
    result=N*i
```

```
    print(N, "*", i, "=", result)
```

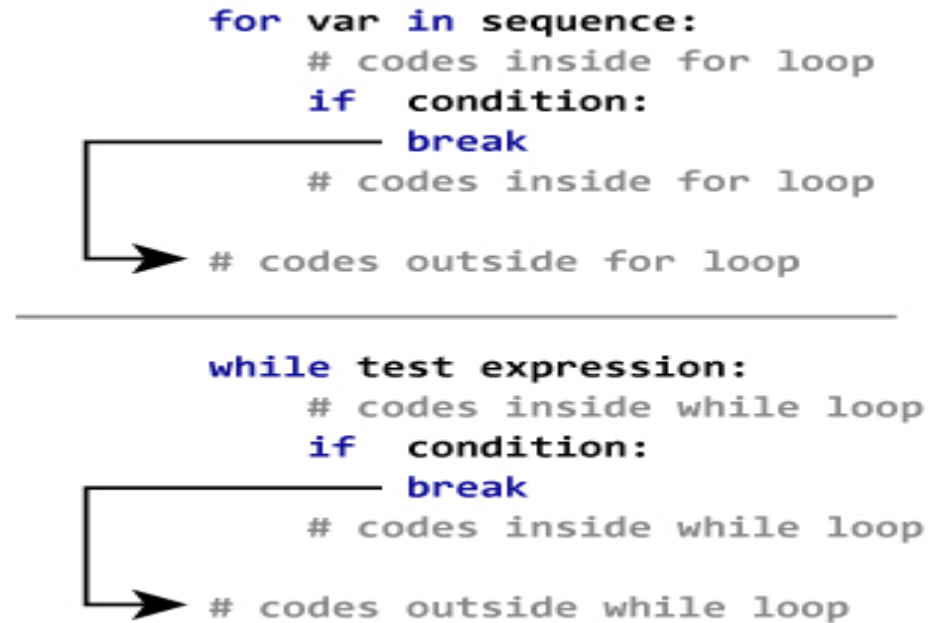
Control Flow Statements

Selection (Unconditional)

- It is used **to transfers the control** from **one part** of the program to **another without any condition**.
- Python supports **three types** of unconditional control statements:
 - break
 - continue
 - pass

Selection (Unconditional): break

- **Break statement** is used to **terminate the execution** of the **nearest enclosing loop** in which it appears.
- When compiler encounters a break statement, the **control passes** to the statement **immediately after the body of the loop**.
- **Syntax:** break



Selection (Unconditional): break

Example 1:

""To get N inputs from the user and calculate the sum of the same.

If user enters -1 then stop adding."""

```
N=int(input("Enter the value of N: "))
```

```
i=1
```

```
sum=0
```

```
while i<=N:
```

```
    num=int(input("Enter the user inputs: "))
```

```
    if num== -1:
```

```
        break
```

```
    else:
```

```
        sum=sum+num
```

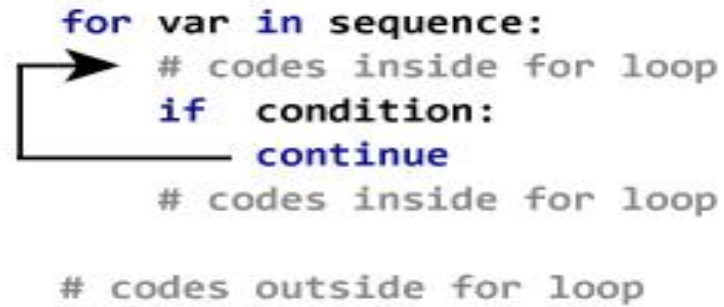
```
    i=i+1
```

```
print('The sum of user input is: ',sum)
```

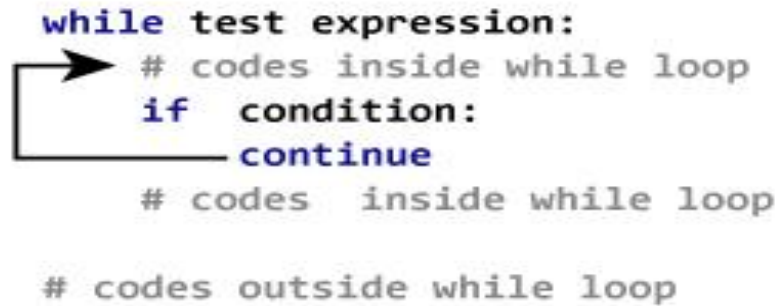
Selection (Unconditional): continue

- When compiler encounters a **continue statement**, then the **rest of the statements in the loop are skipped** and the control is unconditionally transferred to the **next iteration of the loop**.
- **Syntax:** continue

```
for var in sequence:  
    # codes inside for loop  
    if condition:  
        continue  
    # codes inside for loop  
  
# codes outside for loop
```



```
while test expression:  
    # codes inside while loop  
    if condition:  
        continue  
    # codes inside while loop  
  
# codes outside while loop
```



Selection (Unconditional): continue

Example 1:

""To get N inputs from the user and skip if user inputs negative number then calculate the sum of the user inputs""

```
N=int(input("Enter the value of N: "))
i=1
sum=0
while i<=N:
    num=int(input("Enter the user inputs: "))
    i=i+1
    if num<0:
        continue
    else:
        sum=sum+num
print('The sum of user input is: ',sum)
```

Selection (Unconditional): pass

- The **pass statement** is used as a **placeholder for future code**. When the pass statement is executed, **nothing happens**, but you **avoid** getting an **error** when **empty code is not allowed**. **Empty code is not allowed** in loops, function definitions, class definitions, or in if statements.
- **Syntax:** pass

Example 1:

```
a = 33
b = 200
if b > a:
    pass
#raise an error without the pass statement
```

Example 2:

```
for i in range(10)
    pass
#raise an error without the pass statement
```

Iteration: for-else Loop

- **For Else loop statement** is a set of statements that are repeatedly or iteratively executed over a sequence.
- When the condition **becomes False**, then the for loop gets terminated and the **else block will print the False statement** .
- **Syntax:**

```
for variable in sequence:  
    statement  
else:  
    statement
```

Note:

The else block just after for is executed only when the loop is NOT terminated by a break statement.

Iteration: for-else Loop

Example 1:

""To check whether a given number is prime or not

Note: Prime number is a natural number greater than 1 that is divisible only by itself and 1""

```
num = int(input("Enter a number: "))
if num > 1:  # prime numbers are greater than 1
    for i in range(2,num): # check for factors
        if (num % i) == 0:
            print(num,"is not a prime number")
            break
    else: #for-else
        print(num,"is a prime number")
else:
    print(num,"Invalid Input")
```

Iteration: while-else Loop

- **While-else loop statement** is a set of statements that are repeatedly executed until the condition is **True**.
- When the condition **becomes False**, then the while loop gets terminated and the **else block will print the False statement**.
- **Syntax:**

```
while condition :  
    indentedBlock  
else:  
    indentedElseBlock
```

Note:

The else block just after while is executed only when the loop is NOT terminated by a break statement.

Iteration: while-else Loop

Example 1:

#To check whether a given number is prime or not

```
num = int(input("Enter a number: "))
if num > 1: # prime numbers are greater than 1
    i=2
    while i<num:
        if (num % i) == 0:
            print(num,"is not a prime number")
            print(i,"times",num//i,"is",num)
            break
        i=i+1
    else: #while-else
        print(num,"is a prime number")
else:
    print(num,"is not a prime number")
```

Iteration: Nested Loop Statements

- **Nested Loop:** A loop inside another loop is called a nested loop. The number of loops depend on the complexity of a problem.
- Suppose, a loop, **outer loop**, running **n number of times** consists of another loop inside it, **inner loop**, running **m number of times**. Then, for **each execution** of the **outer loop** from **1...n**, the inner loop runs maximum of **m times**.
- **Syntax:**

```
while condition:  
    statement  
while condition:  
    statement
```

```
for variable in sequence:  
    statement  
for variable in sequence:  
    statement
```

Iteration: Nested Loop Statements

Example 1:

#To print the prime numbers between two intervals

```
L= int(input("Enter Lower Limit: "))
U= int(input("Enter Upper Limit: "))
print("The Prime numbers between",L, "and",U,"are:")
for num in range(L, U+1):
    if num > 1: # prime numbers are greater than 1
        for i in range(2,num):
            if (num % i) == 0:
                break
        else: #for-else
            print(num)
```

Iteration: Nested Loop Statements

Example 2:

#To print the multiplication table between two intervals

```
start=int(input("Enter starting Table: "))
end=int(input("Enter ending Table: "))
limit=int(input("Enter Table limit: "))
while start<=end:
    print("MULTIPLICATION TABLE FOR ", start)
    j=1
    while j<=limit:
        print(start, "*", j, "=", start*j)
        j=j+1
    start=start+1
```

Quiz

1. Which of the following are the control structure in python?

a) Selection

b) Iteration

c) Both a and b

d) Sequential

c) Both a and b

2. Leading whitespace at the beginning of a statement is called

a) Indentation

b) Orientation

c) Iteration

d) None of these

a) Indentation

3. Janu wants to check whether a year is leap year or not. To do this what statement she may use?

a) conditional statement

b) Iterative Statement

a) Conditional statement

4. What keyword would you use to add an alternative condition to an if statement?

a) else if

b) if

c)elif

c)pass

c) elif

Quiz

5. What is the output of the following loop?

```
for l in 'smart cliff':  
    if l == 'r':  
        pass  
    print(l, end = ",")
```

a) smart cliff

b) sma

c) sma cliff

d) None of these

d) None of these

Quiz

6. What is the output of the following for loop and range() function?

```
for num in range(-2,-5,-1):  
    print(num, end=",")
```

a) -2,-1,-3,-4

b) -2,-1,0,1,2,3

c) -2,-1,0

d) -2,-3,-4

d) -2,-3,-4

Control Flow Statements

Quiz

7. Function `range(3)` is equivalent to

a) `range(1,3)`

b) `range(0,3)`

c) `range(0,3,1)`

d) `range(1,3,0)`

b)&c)

Control Flow Statements

Quiz

8. Function `range(10,5,-2)` will yield an iterable sequence like

a) [10,8,6]

b) [9,7,5]

c) [6,8,10]

d) [5,7,9]

a) [10,8,6]

THANK YOU